HarvestBox: Planning Tool for a Meal-Kit Company

BUS 43100 Mid-Quarter Update | All data in this project is simulated

1. Problem definition

A meal-kit company like HarvestBox has to decide every week how many kits to prepare for each region and plan type, and in what order its trucks deliver them. Two people own these decisions: the **operations planner** (how much food to prepare) and the **dispatcher** (truck routes). Prepared food cannot be un-made or restocked in time, so every forecast error costs money in one of two ways: too many kits means **food waste**, too few means **stockouts** (customers who do not get their box). The simplest practice, and the one I use as my baseline, is to prepare "same as last week." In my 20-week test that rule wasted 4,290 kits (8.3% of everything ordered) and left 1,245 kits of demand unserved, about **$46,000** under my assumed costs ($9 per wasted kit, $6 per missed kit). On the delivery side, the baseline visits stops in the order they were received.

Contact with reality. So far the project runs on simulated data, and my grounding is secondhand (how meal-kit operations are generally described); the cost figures are my own assumptions. I simulated on purpose: it keeps the scope small and avoids real customer addresses, which are personal data and would need permission and privacy protections. I am now trying to get real data and hope to have it next week (week 7). If I succeed, it will make the work more complex (privacy and anonymization, cleaning messy records, a forecast that no longer matches the patterns I built in, and costs that must be replaced with real figures), so I treat it as an upgrade to the plan and not as a given. Until then, every result is labelled as simulated.

Scope and success. I kept the problem small: 3 regions, 3 plan types, 1 depot, 2–3 trucks, about 40 delivery stops a week. Success means: (a) the forecast has lower error (MAE, MAPE) than simple baselines on the most recent 20 weeks; (b) fewer wasted kits and stockouts at the same service level; (c) fewer miles than in-order routing (not measured yet); and (d) a human approves every decision, since the tool recommends and people decide. The problem itself has not changed since I started; what changed is how demanding I am about fair baselines (see section 3).

2. Approach and rationale

I split the problem into three parts and only one uses machine learning, because the course question is whether the benefits outweigh the costs and risks, not whether AI can be used.

- **Weekly demand forecast: ML** (gradient boosting vs. baselines). Demand has trend, seasonality, holidays and promotions, and there is history to learn from, so this is a natural fit for ML.
- **Inventory quantity: arithmetic** (forecast x (1 + safety buffer)). The planner can understand and adjust one number. ML would add cost and opacity with no benefit.
- **Delivery routes: optimization** (OR-Tools), not ML. It is a well-defined math problem (capacity-limited trucks, one depot); the best answer can be computed, not predicted.

Alternatives and tradeoffs. I compared ML against two non-AI forecasts (same as last week; 4-week average). A bigger safety buffer is another non-AI "fix", but it trades stockouts for more waste. Gradient boosting is less explainable than a moving average, needs data and upkeep, and, as shown below, its advantage is concentrated in promotion and holiday weeks; in ordinary weeks a 4-week average is almost as good. That is where AI may not be worth it. I also treat an optional Claude-written weekly summary as a risk (it can misstate numbers), so a human must check it against the data. I build in Python (pandas, scikit-learn, OR-Tools, Streamlit) with Claude Code, one step at a time, guided by a CLAUDE.md file of fixed rules: small scope, always compare to a baseline, time-based split only, fixed random seed, simulated data labelled everywhere.

3. Learning about AI through the work

- **Instructions shape output.** Writing the hard rules down once (baseline comparison, no random shuffling, seed 42, "SIMULATED DATA" on every chart) made each step come back in a consistent, checkable form. Building in small steps, with a commit after each, let me look at results before moving on.
- **What is happening behind the AI.** I learned that when I ask the AI for something, it is writing Python code behind the scenes and connecting libraries (pandas for the data, scikit-learn for the model, OR-Tools for routes, Streamlit for the dashboard). The real product is that code, which I can read, run and check, not just the AI's explanation of it.

- **A good-looking result needed a second look.** The first comparison said ML cut error by 39% versus "same as last week." Feature importance showed one feature, the 4-week average, dominated (103.8 vs. at most 4.1 for any other). That suggested the gain might come from smoothing, not from ML, so I added a stronger baseline. The gain shrank to about 20% over the 4-week average, which is a more honest headline.
- **Failure modes specific to this problem:** (1) simulated data can flatter the model, because the patterns it learns are the ones I built in; (2) a weak baseline inflates the benefit; (3) promotion and holiday weeks are the hardest (baseline error about 56 orders vs. 22 in normal weeks); (4) fluent AI explanations and summaries sound sure even when a number is wrong.

4. Checking the work

Standard of "good": beat honest baselines on data the model never saw. I train on the past and test on the last 20 weeks (2025-08-11 to 2025-12-22), then replay those weeks through the inventory rule.

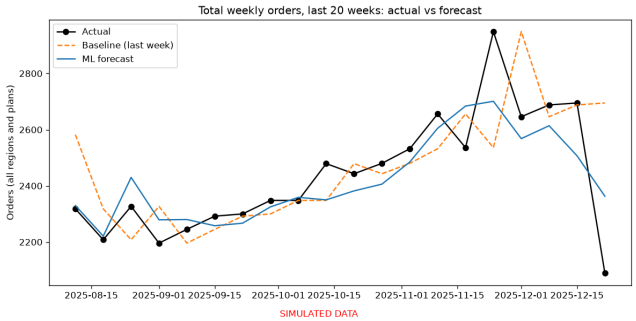
- **Same as last week:** MAE 27.45, MAPE 10.05%, waste 4,290 kits, stockouts 1,245 kits, cost $46,080.
- **4-week average:** MAE 20.88, MAPE 7.60%, waste 3,292 kits, stockouts 1,082 kits, cost $36,120.
- **ML (gradient boosting):** MAE 16.63, MAPE 5.90%, waste 2,833 kits, stockouts 691 kits, cost $29,643.

Inventory figures use a 5% safety buffer. Cost uses my assumed $9/kit waste and $6/kit stockout.

What I found, problems included:

- ML beats both baselines overall, but **in normal weeks the 4-week average is slightly better** (MAE 14.6 vs. 15.1). ML wins in promo weeks (14.9 vs. 52.1) and holiday weeks (30.7 vs. 58.7), where it can use the promo and holiday flags.
- At an equal 98% fill rate, ML needs a 4% buffer (waste 2,488 kits), the 4-week average 6% (3,651) and last-week 8% (5,384).
- Across 10 other simulated datasets ML beat both baselines every time (mean MAPE 6.65% vs. 7.80% and 9.65%), but only narrowly once (6.68 vs. 6.74). ML was cheapest under three different waste/stockout cost settings.
- **Limit:** all of this is on data I generated, so it shows that the model learns the patterns I put in, not that it works on real demand.

Plan for fuller evaluation: measure routing against in-order and hand-checked routes; check every number in any AI summary against the data; talk to someone with real operations or logistics experience about whether the forecast error and costs are realistic; and run the dashboard's approval flow end to end.



Total weekly orders, last 20 weeks: actual vs. baseline vs. ML (simulated data)

5. Work completed and plan for weeks 7–10

Done (code and results in the repo): simulated data (1,404 weekly order rows, about 6,240 delivery stops over 3 years); forecast with baseline, ML, error by week type and feature importance; inventory simulation with an adjustable safety buffer and a trade-off chart, plus a breakdown by region and plan; dashboard v1 (demand explorer with week view and history); extra checks (stronger baseline, equal-service comparison, 10 seeds, cost sensitivity). **Not done yet:** routing, the planner and dispatcher Approve buttons, KPI cards, the optional summary.

- **Week 7:** try to get real data (my goal is to have it this week) and check what it contains and whether it can be used privately; build routing: OR-Tools with capacity-limited trucks vs. in-order baseline; compare miles and cost; return coordinates for the map.
- **Week 8:** finish the dashboard: next-week forecast, inventory recommendation + planner Approve, route map + dispatcher Approve, KPI cards, simulated-data footer.

- **Week 9:** evaluation: if the real data arrived, test the forecast on it and compare with the simulated results (if not, keep trying but do not wait for it); talk to someone with ops/logistics experience and adjust assumptions; optional Claude summary with a number check; fix bugs.
- **Week 10:** final report (risks, results, screenshots), README, rerun everything from the seed, record the video.

Risks and what I will do if they happen:

- **Routing takes longer than planned:** use a smaller instance, or keep the baseline plus a simple improvement and say so honestly.
- **Real data is hard to get, or makes the pipeline more complex:** keep the simulated version as the main result and label it clearly; if real data does not arrive in week 7, or arrives late, test it only on the forecast step and report it as an extra.
- **AI summary misstates numbers:** show the source numbers beside it, or drop the feature.
- **Time runs short in finals week:** freeze new features at the end of week 9; spend week 10 on the report and checking.